МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий інститут прикладного системного аналізу Кафедра системного проєктування

Розрахунково графічна робота

на тему: “Множення матриці на вектор”

Мета РГР: “Набуття навичок програмування додатків розподілених обчислень і оцінки характеристик їх ефективності”

Виконав:
студент групи КН-51мп
Сидько Нікіта Сергійович

Київ – 2025

Зміст

1. Вступ.....3
2. Теоретичні відомості.....4
2.1. Паралельні та розподілені обчислення.....4
2.2. Основи MPI та модель процесів.....4
2.3. Самостійно плануючі алгоритми (Self Scheduling).....5
3. Реалізація алгоритму множення матриці на вектор.....6
3.1. Опис послідовного та паралельного алгоритмів.....6
3.2. Основні модулі програми MPI.....7
4. Експериментальна частина.....8
4.1. Методика тестування.....8
4.2. Результати тестування.....9
4.3. Графічний аналіз результатів.....9
5. Аналіз ефективності алгоритму.....10
6. Висновок.....11
7. Список використаних джерел.....12
Додаток А.....13

1. Вступ

Розвиток сучасних інформаційних технологій супроводжується постійним зростанням обсягів даних та підвищенням вимог до швидкості їх обробки. У зв’язку з цим особливої важливості набувають методи паралельних та розподілених обчислень, які дозволяють прискорити виконання задач за рахунок використання декількох процесорів або обчислювальних вузлів одночасно. Такі методи широко застосовуються у наукових дослідженнях, інженерних розрахунках, моделюванні фізичних процесів, машинному навчанні та інших галузях, де потрібні значні обчислювальні ресурси.

Одним із найпоширеніших інструментів для створення паралельних програм є стандарт MPI (Message Passing Interface), що забезпечує взаємодію між процесами шляхом обміну повідомленнями. MPI дозволяє створювати масштабовані додатки, здатні працювати як на багатоядерних процесорах, так і на кластерних системах. На відміну від моделей паралелізму, що використовують спільну пам’ять, MPI забезпечує чіткий розподіл пам’яті між процесами, що підвищує надійність і гнучкість програм.

У даній розрахунково-графічній роботі розглядається задача множення матриці на вектор, яка є однією з базових операцій лінійної алгебри. Незважаючи на простоту формулювання, її ефективне виконання має важливе практичне значення, оскільки ця операція широко застосовується у чисельних методах, системах моделювання, обробці сигналів і машинному навчанні. Особливу увагу приділено самостійно плануючому алгоритму (Self-Scheduling Algorithm), який забезпечує динамічний розподіл роботи між процесами під час виконання множення. На відміну від статичного розподілу, де кожному процесу призначається фіксована частина задачі, динамічний підхід дозволяє уникнути ситуацій, коли одні процеси простоюють, а інші перевантажені.

Метою роботи є розробка та експериментальне дослідження самостійно плануючого алгоритму множення матриці на вектор на основі MPI.

2. Теоретичні відомості

2.1. Паралельні та розподілені обчислення

Паралельні обчислення передбачають одночасне виконання кількох операцій з метою прискорення обробки даних або вирішення задачі. Основна ідея полягає у розбитті великої задачі на менші підзадачі, які можуть виконуватися незалежно одна від одної. У сучасних комп’ютерах паралелізм реалізується через багатоядерні процесори, графічні прискорювачі (GPU) та кластери.

Розподілені обчислення є частковим випадком паралельних обчислень, але з тією відмінністю, що процеси виконуються на різних вузлах мережі, кожен з яких має власну пам’ять. Взаємодія між такими процесами відбувається шляхом передачі повідомлень, що робить їх більш масштабованими, але водночас створює додаткові витрати на комунікацію. Розподілені системи дозволяють вирішувати задачі великої розмірності, які неможливо обробити на одному фізичному комп’ютері.

У задачах лінійної алгебри, таких як множення матриць чи розв’язання систем рівнянь, паралелізм дозволяє значно зменшити час виконання, оскільки обчислення для різних частин даних можуть виконуватися незалежно.

2.2. Основи MPI та модель процесів

MPI (Message Passing Interface) – це стандарт взаємодії між процесами у розподілених і багатопроцесорних системах. Його головною ідеєю є явний обмін повідомленнями, що дозволяє програмісту контролювати логіку розподілу даних та передачі результатів.

Модель MPI базується на таких поняттях:

1. Процес (process) – незалежна програма, що має власний адресний простір і виконується паралельно з іншими процесами.
2. COMM_WORLD – глобальний комунікатор, що містить усі процеси MPI-програми.
3. rank – унікальний номер процесу всередині комунікатора.
4. size – загальна кількість процесів у комунікаторі.

MPI забезпечує два ключові механізми взаємодії:

1. Точка-точка (point-to-point). Передача повідомлень між конкретними процесами (MPI_Send, MPI_Recv).
2. Колективні операції (collective operations). Передача даних між усіма процесами, наприклад MPI_Bcast (розсилка), MPI_Reduce (зведення результатів), MPI_Scatter та MPI_Gather.

Важливо, що MPI-програма складається з декількох копій одного й того самого коду, які виконуються в режимі SPMD (Single Program, Multiple Data). Логіка програми визначає роль кожного процесу – наприклад, процес з rank=0 може бути master-процесом.

2.3. Самостійно плануючі алгоритми (Self Scheduling)

У паралельних обчисленнях однією з ключових задач є забезпечення рівномірного розподілу навантаження між процесами. Якщо використовується статичний розподіл, коли кожному процесу заздалегідь призначається фіксована частина роботи, це може призвести до ситуації, коли окремі процеси завершують свої обчислення раніше за інших і вимушено простоюють. Такий підхід не забезпечує повного використання доступних обчислювальних ресурсів і знижує ефективність паралелізації.

Самостійно плануючі алгоритми передбачають динамічний розподіл роботи між процесами. У цій схемі процеси-робітники самі ініціюють запит до головного процесу після завершення поточного завдання. У відповідь на цей запит головний процес видає наступний елемент роботи. Коли всі елементи вже оброблено, робітникові надсилається повідомлення про завершення. Таким чином, кожен процес отримує нове завдання лише після того, як виконає попереднє, і система автоматично вирівнює навантаження.

У задачі множення матриці на вектор такий підхід означає, що кожен процес у будь-який момент працює лише з одним рядком матриці. Після завершення обчислення скалярного добутку цього рядка на вектор результат

передається головному процесу, після чого робітник знову запитує нове завдання. Це дозволяє оптимально використовувати всі доступні процеси незалежно від того, наскільки швидко вони виконують свої частини роботи.

Завдяки цьому динамічний розподіл забезпечує мінімізацію простоїв, рівномірніше завантаження процесів та кращу масштабованість алгоритму порівняно зі статичними підходами. Він є особливо ефективним у випадках, коли обсяг обчислень для різних частин задачі є нерівномірним або коли продуктивність обчислювальних ресурсів відрізняється між собою.

3. Реалізація алгоритму множення матриці на вектор

3.1. Опис послідовного та паралельного алгоритмів

У послідовному алгоритмі множення матриці на вектор виконується обчислення скалярного добутку кожного рядка матриці на вектор. Цей процес виконується в одному потоці, тому час виконання пропорційний кількості рядків.

У паралельному варіанті це обчислення розподіляється між кількома процесами. У самостійно плануючому алгоритмі кожен процес динамічно отримує наступний доступний рядок матриці від головного процесу, обчислює відповідний елемент вектора результату та повертає його. Такий підхід забезпечує рівномірний розподіл навантаження та мінімізує простої.

3.2. Основні модулі програми MPI

Функція обчислення одного елемента результату наведена нижче (див. рис.1)

```
def compute_row(A_row, x):
    return np.dot(A_row, x)
```

Рис.1 Функція обчислення

Ця функція виконується робітниками і відповідає за обчислення одного елемента вектора результату.

Далі йде реалізація роботи головного процесу (master) (див.рис.2)

```python
def master_process(N):
    A, x = generate_data(N)
    y = np.zeros(N)

    comm.bcast(x, root=0)

    next_row = 0
    active = size - 1
    start = MPI.Wtime()

    while active > 0:
        status = MPI.Status()
        comm.recv(source=MPI.ANY_SOURCE, tag=REQUEST, status=status)
        worker = status.Get_source()

        if next_row < N:
            comm.send(next_row, dest=worker)
            comm.send(A[next_row], dest=worker)

            result = comm.recv(source=worker)
            y[next_row] = result
            next_row += 1
        else:
            comm.send(STOP, dest=worker)
            active -= 1

    end = MPI.Wtime()
    print(f"[MPI {size} processes] Matrix {N}×{N} → Time: {end - start:.4f}s | Correct: {np.allclose(y, A.dot(x))}")
```

Рис. 2 master_process

Головний процес формує дані, розсилає вектор, видає процесам наступний рядок матриці та приймає результати.

Потім йде реалізація процесів-робітників (worker) (див.рис.3) наведений нижче

```python
def worker_process():
    x = comm.bcast(None, root=0)

    while True:
        comm.send(None, dest=0, tag=REQUEST)
        row_index = comm.recv(source=0)

        if row_index == STOP:
            break

        A_row = comm.recv(source=0)
        result = compute_row(A_row, x)
        comm.send(result, dest=0)
```

Рис.3 worker_process

Робітники запитують роботу, отримують рядки матриці, виконують обчислення та повертають результат.

Основний блок запуску програми наведений нижче (див.рис.4) визначає, який процес буде виконувати роль головного, а які - роль робітників.

```
if __name__ == "__main__":
    N = 100

    if rank == 0:
        master_process(N)
    else:
        worker_process()
```

Рис.4 Запуск програми

Програма запускається через команду: mpirun -np N python script.py, де N це кількість процесів.

Більш детальний опис коду та програми в цілому наведений у Додатку А (посилання на Google Colab, де попукнто описана реалізація роботи в цілому)

4. Експериментальна частина

4.1. Методика тестування

Для оцінювання ефективності самостійно плануючого MPI-алгоритму було проведено серію експериментів з різними розмірами задачі та різною кількістю процесів. Тестування виконувалося у середовищі Google Colab, де всі MPI-процеси запускаються на одному фізичному процесорному ядрі. Незважаючи на це обмеження, середовище дозволяє відтворити поведінку паралельного алгоритму та оцінити вплив створення додаткових процесів на час виконання. Алгоритм тестувався для трьох розмірів матриць:

1. 1000×1000,
2. 5000×5000,
3. 10000×10000.

Для кожного розміру були виконані вимірювання при різній кількості процесів: 2, 4, 6, 8 та 10 процесів. У кожному експерименті фіксувався загальний час

виконання алгоритму, а також перевірялася коректність отриманого результату шляхом порівняння з послідовним множенням матриці на вектор.

4.2. Результати тестування

Нижче наведено таблицю (див.табл.1), яка містить отримані експериментальні дані.

Кількість процесів	**1000×1000**	**5000×5000**	**10000×10000**
2 процеси	0.0884 с	0.5829 с	2.6306 с
4 процеси	0.1739 с	0.7971 с	1.8003 с
6 процесів	0.4203 с	1.0987 с	3.9020 с
8 процесів	0.3022 с	1.7820 с	3.5732 с
10 процесів	0.3982 с	2.9551 с	4.9984 с

Таблиця 1. Час виконання алгоритму для різної кількості процесів (у секундах)

4.3. Графічний аналіз результатів

Для наочного відображення залежності часу виконання від кількості процесів було побудовано лінійні графіки для кожного розміру матриці. Графіки показують, як змінюється продуктивність алгоритму при збільшенні числа процесів.

Для матриці 1000×1000 час виконання змінюється нерівномірно, оскільки обсяг роботи невеликий, а накладні витрати MPI мають суттєвий вплив.

Для 5000×5000 та 10000×10000 спостерігається більш виражена закономірність: час виконання зменшується при збільшенні кількості процесів до певного моменту, після чого починає зростати. Це пов’язано з тим, що кількість процесів зростає, але доступні обчислювальні ресурси фізично не додаються, оскільки всі процеси виконуються на одному ядрі.

На графіках для кожної точки позначено фактичний час виконання, що дозволяє легко порівнювати результати.

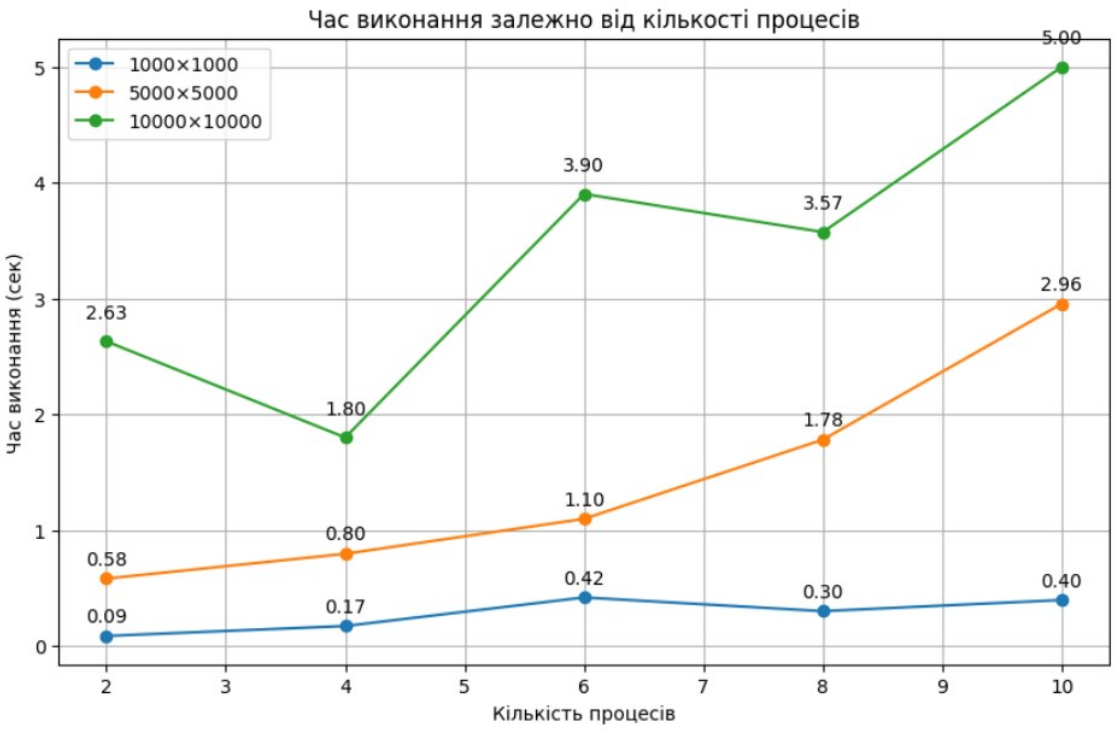


Рис.5 Графічний аналіз результатів

5. Аналіз ефективності алгоритму

Отримані результати демонструють, що ефективність самостійно плануючого алгоритму множення матриці на вектор залежить як від кількості процесів, так і від розмірності задачі. Для малих матриць паралельне виконання не забезпечує помітного прискорення через те, що накладні витрати на комунікацію між процесами є порівнянними або навіть вищими за власне час обчислення. У таких випадках збільшення числа процесів іноді призводить до погіршення часу виконання.

Для більших матриць частка обчислень зростає, тому паралельний алгоритм працює стабільніше та демонструє кращу масштабованість. Проте у середовищі Google Colab всі процеси виконуються на одному фізичному ядрі, що обмежує можливість реального прискорення. Незважаючи на це, проведені експерименти дозволяють оцінити поведінку алгоритму, порівняти різні конфігурації та зробити висновки щодо впливу динамічного розподілу роботи на продуктивність.

6. Висновок.

У ході виконання розрахунково графічної роботи було реалізовано та досліджено самостійно плануючий алгоритм множення матриці на вектор з використанням бібліотеки MPI. Дослідження дозволило проаналізувати особливості організації паралельних обчислень у моделі обміну повідомленнями та оцінити вплив динамічного розподілу завдань на ефективність виконання програм. Реалізований алгоритм демонструє характерні риси схем з динамічним балансуванням навантаження, при яких кожен робітник самостійно запитує нове завдання одразу після завершення попереднього. Такий підхід забезпечує рівномірніше використання процесів і зменшує можливість простоїв, що особливо важливо для задач з нерівномірною складністю окремих обчислень.

Проведені експерименти показали, що час виконання суттєво залежить від розміру матриці та кількості процесів. Для малих матриць обсяг обчислень є недостатнім для досягнення прискорення, оскільки накладні витрати на комунікацію між процесами становлять значну частину загального часу. У таких випадках збільшення кількості процесів не призводить до зменшення часу виконання та іноді може давати протилежний ефект. Для більших матриць частка обчислень є значно більшою, тому алгоритм працює стабільніше, а вплив комунікацій зменшується. Це підтверджує важливість врахування співвідношення обчислень і обміну даними під час вибору схеми паралелізації.

Проведена робота продемонструвала, що самостійно плануючий підхід до розподілу завдань є гнучким і універсальним методом організації паралельних обчислень. Він дозволяє досягти кращого балансування навантаження у порівнянні зі статичними методами та адаптується до характеристик обчислювального середовища. Реалізація та експериментальна перевірка алгоритму дали можливість глибше зрозуміти принципи побудови паралельних програм у моделі MPI та проаналізувати фактори, що впливають на їхню ефективність.

7. Список використаних джерел

1. Gropp, W., Thakur, R., & Lusk, E. Using MPI: Portable Programming with the Message Passing Interface. MIT Press, 2014.
2. Pacheco, P. Parallel Programming with MPI. Morgan Kaufmann Publishers, 1997.
3. Quinn, M. J. Parallel Programming in C with MPI and OpenMP. McGraw-Hill, 2004.
4. Snir, M., Otto, S., Huss-Lederman, S., Walker, D., & Dongarra, J. MPI: The Complete Reference. MIT Press, 1998.
5. Dongarra, J., Duff, I., Sorensen, D., & van der Vorst, H. Numerical Linear Algebra for High-Performance Computing. SIAM, 1998.
6. mpi4py Documentation. MPI for Python, Official Documentation. Available at: https://mpi4py.readthedocs.io/
7. Barrett, R., Berry, M., Chan, T. et al. Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods. SIAM, 1994. Available at: https://www.netlib.org/templates/
8. Wilkinson, B., & Allen, M. Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers. Prentice Hall, 2005.
9. Foster, I. Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering. Addison-Wesley, 1995.
10. Матеріали лекцій з дисципліни «Розподілені обчислення та інтернет-технології», кафедра комп’ютерних наук.

Додаток А

https://colab.research.google.com/drive/18miht7hFU2q9jsTWARV7QHKMoFmOozAU?usp=sharing